

Deploying React Apps

Easy Guide

Contents

1	Deployment Steps	2
1.1	What is Deployment?	2
1.2	The Journey	2
1.3	Deployment Platforms	2
1.4	Step-by-Step Deployment	2
1.4.1	Step 1: Prepare Code	2
1.4.2	Step 2: Build for Production	3
1.4.3	Step 3: Create Account	3
1.4.4	Step 4: Connect Your Repository	3
1.4.5	Step 5: Configure Settings	3
1.4.6	Step 6: Deploy	3
1.4.7	Step 7: Get URL	3
1.5	Common Platforms	3
1.5.1	Vercel (Best for Next.js)	3
1.5.2	Netlify (Best for Simple Apps)	4
1.6	Deployment Checklist	4
2	Understanding Lazy Loading	5
2.1	What is Lazy Loading?	5
2.2	Why Care?	5
2.3	Before Lazy Loading	5
2.4	After Lazy Loading	5
2.5	What Can Be Lazy Loaded?	6
2.6	How It Works	6
2.7	Bundle Size Impact	6
3	Adding Lazy Loading	7
3.1	React Lazy Loading	7
3.2	What's Happening	8
3.3	With React Router	8
3.4	Better: Reusable Suspense	8
3.5	Lazy Loading Images	8
3.6	Lazy Loading Libraries	9
3.7	Testing Lazy Loading	9

4	Building the Code For Production	10
4.1	Development vs Production	10
4.1.1	Development	10
4.1.2	Production	10
4.2	Build Command	10
4.3	What Happens	10
4.4	File Size Reduction	11
4.5	Build Output	11
4.6	Vite Build Example	11
4.7	Environment Variables in Build	12
4.8	Building Locally to Test	12
4.9	Common Build Issues	12
4.9.1	Build Fails	12
4.9.2	Large Bundle	12
5	Server-side Routing & Required Configuration	13
5.1	Client-Side vs Server-Side Routing	13
5.1.1	Client-Side (React Router)	13
5.1.2	Server-Side (Traditional)	13
5.2	The Problem	13
5.3	The Solution: Catch-All Routes	14
5.4	Configure for Netlify	14
5.5	Configure for Vercel	14
5.6	Configure for Apache	14
5.7	Configure for Nginx	15
5.8	Why This Works	15
5.9	Testing Configuration	15
5.10	Common Configuration Issues	15
5.10.1	404 on Page Refresh	15
5.10.2	CSS/JS Not Loading	16
5.10.3	Infinite Redirect Loop	16
5.11	Testing Before Deploy	16

Chapter 1

Deployment Steps

1.1 What is Deployment?

Taking your React app from your computer and putting it on the internet so everyone can use it.

1.2 The Journey

Your Computer (Development)
↓
Build for Production
↓
Upload to Server
↓
Public Internet
↓
Users can access

1.3 Deployment Platforms

Popular services for hosting React apps:

- **Vercel** - Made by Next.js creators, very easy
- **Netlify** - Simple, great for static sites
- **GitHub Pages** - Free, if you don't need a backend
- **Heroku** - Older but still popular
- **AWS** - Powerful but complex
- **Firebase** - Google's hosting, easy setup

1.4 Step-by-Step Deployment

1.4.1 Step 1: Prepare Code

- Make sure everything works locally

- Fix all bugs and errors
- Test thoroughly

1.4.2 Step 2: Build for Production

```
npm run build
```

Creates an optimized, minified version of your app.

1.4.3 Step 3: Create Account

Sign up on your chosen platform (Vercel, Netlify, etc.)

1.4.4 Step 4: Connect Your Repository

- Upload code to GitHub
- Connect GitHub to hosting platform
- Platform pulls code automatically

1.4.5 Step 5: Configure Settings

- Set build command: `npm run build`
- Set output directory: `dist` or `build`
- Add environment variables if needed

1.4.6 Step 6: Deploy

Platform automatically builds and deploys your app.

1.4.7 Step 7: Get URL

Platform gives you a public URL like:

```
https://myapp.vercel.app  
https://myapp.netlify.app
```

1.5 Common Platforms

1.5.1 Vercel (Best for Next.js)

1. Go to vercel.com
2. Click “Import Project”
3. Select GitHub repository
4. Click “Deploy”
5. Done! Automatic deployments on every push

1.5.2 Netlify (Best for Simple Apps)

1. Go to netlify.com
2. Click “New site from Git”
3. Select GitHub
4. Choose repository
5. Click “Deploy site”
6. Done!

1.6 Deployment Checklist

- App works locally
- All bugs fixed
- Tests pass
- Environment variables set
- API endpoints configured
- Code committed to GitHub
- Deployment platform configured
- Build settings correct
- Database (if needed) set up

Key Concept: Deployment is moving your local app to the internet!

Chapter 2

Understanding Lazy Loading

2.1 What is Lazy Loading?

Loading code only when you need it, not all at the start.

2.2 Why Care?

- Faster initial load time
- Users see something quickly
- Rest loads in background
- Better performance
- Smaller initial bundle

2.3 Before Lazy Loading

```
App loads
↓
ALL code downloaded (200KB)
↓
ALL code processed
↓
Takes 5 seconds
↓
User sees app
```

2.4 After Lazy Loading

```
App loads
↓
Essential code downloaded (50KB)
↓
Takes 1 second
```

↓
User sees app immediately
↓
Other code loads in background

2.5 What Can Be Lazy Loaded?

- Pages that users don't always visit
- Heavy components
- Third-party libraries
- Images and media
- Styles

2.6 How It Works

```
routes/  
  Home.jsx (loaded immediately)  
  Dashboard.jsx (loaded on demand)  
  Admin.jsx (loaded on demand)
```

When user visits /dashboard:

1. Check if Dashboard code is loaded
2. If not, download it
3. While downloading, show spinner
4. When ready, show component

2.7 Bundle Size Impact

Without lazy loading:
app.js = 500KB (all code)

With lazy loading:
app.js = 150KB (core only)
dashboard.chunk.js = 100KB (loaded later)
admin.chunk.js = 150KB (loaded later)
home.chunk.js = 100KB (loaded later)

Much better! User gets app 3x faster.

Benefit: Lazy loading makes apps load faster!

Chapter 3

Adding Lazy Loading

3.1 React Lazy Loading

Use React's `lazy()` and `Suspense`:

```
import { lazy, Suspense } from 'react';

const Dashboard = lazy(() => import('./pages/Dashboard'));
const Admin = lazy(() => import('./pages/Admin'));

function App() {
  return (
    <Routes>
      <Route path="/" element={<Home />} />
      <Route
        path="/dashboard"
        element={
          <Suspense fallback={<Spinner />}>
            <Dashboard />
          </Suspense>
        }
      />
      <Route
        path="/admin"
        element={
          <Suspense fallback={<Spinner />}>
            <Admin />
          </Suspense>
        }
      />
    </Routes>
  );
}
```

3.2 What's Happening

1. `lazy()` imports component on demand
2. `Suspense` shows fallback while loading
3. When component loads, `Suspense` renders it

3.3 With React Router

```
import { lazy } from 'react';
import { Suspense } from 'react';

const Dashboard = lazy(() => import('./pages/Dashboard'));

const routes = [
  {
    path: '/dashboard',
    element: (
      <Suspense fallback={<LoadingSpinner />}>
        <Dashboard />
      </Suspense>
    )
  }
];
```

3.4 Better: Reusable Suspense

```
function LazyRoute({ component: Component }) {
  return (
    <Suspense fallback={<LoadingSpinner />}>
      <Component />
    </Suspense>
  );
}

const routes = [
  { path: '/dashboard', element: <LazyRoute component={Dashboard} /> },
  { path: '/admin', element: <LazyRoute component={Admin} /> }
];
```

3.5 Lazy Loading Images

```

```

Browser automatically lazy loads images!

3.6 Lazy Loading Libraries

```
// Instead of this (loads everything):
import { Chart } from 'chart-library';

// Do this (load only when needed):
const Chart = lazy(() => import('chart-library'));

// Use inside Suspense
<Suspense fallback={<p>Loading chart...</p>}>
  <Chart />
</Suspense>
```

3.7 Testing Lazy Loading

```
// Slow 3G simulation in Chrome DevTools
// Network tab → Throttling → Slow 3G
// Navigate to page and see loading spinner
```

Implementation: Use `lazy()` + `Suspense` for components!

Chapter 4

Building the Code For Production

4.1 Development vs Production

4.1.1 Development

- Large file size (easier to debug)
- Slow performance
- All source code visible
- Helpful error messages
- Used for testing locally

4.1.2 Production

- Small file size (minified)
- Fast performance
- Code compressed
- Less helpful error messages (for security)
- Used on internet

4.2 Build Command

```
npm run build
```

Creates an optimized version ready for internet.

4.3 What Happens

1. All code merged together
2. Code minified (removes extra spaces)

3. Unused code removed
4. CSS optimized
5. Images compressed
6. Creates `dist/` or `build/` folder

4.4 File Size Reduction

Before build:

```
app.jsx = 50KB
styles.css = 20KB
Total = 70KB
```

After build:

```
app.min.js = 15KB
styles.min.css = 5KB
Total = 20KB
```

Result: 70% smaller! Much faster!

4.5 Build Output

After running `npm run build`, you get a `dist/` folder:

```
dist/
  index.html
  assets/
    app-abc123.js
    styles-def456.css
    vendor-ghi789.js
  favicon.ico
```

This entire folder is what you deploy!

4.6 Vite Build Example

```
$ npm run build
```

```
vite v4.0.0 building for production...
  234 modules transformed.
dist/index.html                0.45 kB
dist/assets/app-abc123.js      15.32 kB
dist/assets/styles-def456.css  5.12 kB

built in 2.34s
```

4.7 Environment Variables in Build

```
// .env.production
VITE_API_URL=https://api.example.com
VITE_ENV=production

// In code
const apiUrl = import.meta.env.VITE_API_URL;
```

Build replaces `VITE_API_URL` with `actualURL`.

4.8 Building Locally to Test

```
# Build for production
npm run build

# Serve the build locally
npm install -g serve
serve -s dist

# Visit http://localhost:3000
```

Test production build before deploying!

4.9 Common Build Issues

4.9.1 Build Fails

Error: Cannot find module 'xyz'

Fix:

1. Check imports are correct
2. Run `npm install`
3. Try again

4.9.2 Large Bundle

Check bundle size:
`npm install -g webpack-bundle-analyzer`

Solutions:

- Add lazy loading
- Remove unused packages
- Use tree-shaking

Process: Build once, deploy `dist/` folder to internet!

Chapter 5

Server-side Routing & Required Configuration

5.1 Client-Side vs Server-Side Routing

5.1.1 Client-Side (React Router)

- Browser handles routing
- Single HTML file
- Fast navigation (no reload)
- Works offline (if configured)

5.1.2 Server-Side (Traditional)

- Server handles routing
- Different HTML files per route
- Page reloads on navigation
- SEO friendly (outdated, not needed now)

5.2 The Problem

When user visits a URL directly (not through link):

User types: `https://myapp.com/dashboard`

Server looks for:
`/dashboard/index.html`
`/dashboard.html`

Not found! Error 404

React Router app exists as single `index.html` file, but server can't find other routes.

5.3 The Solution: Catch-All Routes

Tell server: ``All unknown routes go to index.html``
React Router then handles the routing.

5.4 Configure for Netlify

Create netlify.toml in project root:

```
[build]
  command = "npm run build"
  publish = "dist"

[[redirects]]
  from = "/*"
  to = "/index.html"
  status = 200
```

This tells Netlify to serve index.html for all routes.

5.5 Configure for Vercel

Create vercel.json in project root:

```
{
  "buildCommand": "npm run build",
  "outputDirectory": "dist",
  "rewrites": [
    {
      "source": "/(.*)",
      "destination": "/index.html"
    }
  ]
}
```

5.6 Configure for Apache

Create .htaccess in public/ folder:

```
<IfModule mod_rewrite.c>
  RewriteEngine On
  RewriteBase /

  # If file doesn't exist, serve index.html
  RewriteCond %{REQUEST_FILENAME} !-f
  RewriteCond %{REQUEST_FILENAME} !-d
  RewriteRule ^ index.html [QSA,L]
</IfModule>
```


5.7 Configure for Nginx

In `nginx.conf`:

```
server {  
  location / {  
    try_files $uri $uri/ /index.html;  
  }  
}
```

5.8 Why This Works

User visits: `/dashboard`

Without configuration:

Server: Looking for `/dashboard` file... Not found! 404 Error

With configuration:

Server: Looking for `/dashboard` file... Not found

Server: OK, serve `/index.html` instead

Browser: Gets `index.html`

React: Router checks URL (`/dashboard`)

React: Finds matching route, renders Dashboard

Result: Works perfectly!

5.9 Testing Configuration

After deploying:

1. Visit app home page
2. Click link to navigate
3. Reload page (F5)
4. Type URL directly in browser
5. All should work!

5.10 Common Configuration Issues

5.10.1 404 on Page Refresh

Missing server-side routing configuration.

Fix: Add catch-all route configuration.

5.10.2 CSS/JS Not Loading

Check asset paths. If app is in subdirectory:

In index.html or build config:
`<base href="/subdirectory/" />`

Or in package.json:
`"homepage": "https://example.com/subdirectory"`

5.10.3 Infinite Redirect Loop

Configuration is redirecting too much.

Check: Catch-all rule shouldn't redirect actual files (/index.html, images, etc.)

5.11 Testing Before Deploy

Build and test locally:

```
npm run build
npx serve -s dist
```

Test these:

1. Visit `http://localhost:3000` (home page)
2. Click navigation link
3. Press F5 to refresh
4. Type URL directly in browser

All should work!

Configuration: Server-side routing config tells server to serve React!

Quick Summary

Deployment Process

1. Build: `npm run build`
2. Create account on Vercel/Netlify/etc
3. Connect GitHub repository
4. Configure build settings
5. Deploy (platform does it automatically)
6. Get public URL
7. Share with world!

Popular Platforms

Platform	Best For	Price
Vercel	Next.js, React	Free with paid tiers
Netlify	Static sites, React	Free with paid tiers
GitHub Pages	Free hosting	Free
Firebase	Quick deployment	Free tier available
AWS	Scalability	Pay as you go

Optimization Techniques

1. Lazy load pages not immediately needed
2. Lazy load heavy images
3. Code splitting for large libraries
4. Remove unused code
5. Minify CSS and JS
6. Use CDN for static assets

Server Configuration

- Set build command: `npm run build`
- Set output directory: `dist`
- Add catch-all routing (`index.html`)
- Set environment variables
- Enable HTTPS

Deployment Checklist

1. Code works locally
2. All tests pass
3. Environment variables set
4. Build succeeds (`npm run build`)
5. Production build works locally
6. Server routing configured
7. Connected to platform
8. Deployed successfully
9. Test live app
10. Share with users!

Common Issues & Fixes

Issue	Fix
404 on page refresh	Add catch-all route config
Large bundle	Add lazy loading
Build fails	Check for errors, run <code>npm install</code>
API calls fail	Check API URL in <code>.env</code>
Styles missing	Check asset paths

Best Practices

1. Use Vercel or Netlify (easiest)
2. Lazy load pages and heavy components
3. Test production build locally first
4. Configure server-side routing

5. Use environment variables for secrets
6. Monitor performance after deploy
7. Set up automatic deployments (on push)
8. Use custom domain (optional)

Performance Tips

1. Aim for bundle < 200KB
2. First load < 3 seconds
3. Lazy load non-critical code
4. Use images wisely (compress, lazy load)
5. Monitor with tools like Lighthouse

Tools Useful

- Vercel CLI - Deploy from command line
- Netlify CLI - Deploy from command line
- Bundle Analyzer - See what's in bundle
- Lighthouse - Performance testing
- Chrome DevTools - Network debugging

Deployment doesn't have to be complicated - choose a platform and let it handle it!